

Midnight Isogenies 🌕

OIST Workshop 2026 (TP25IC), 18th February 2026

Giacomo Pope

This Morning / Tonight's Plan

- Talk about some upcoming joint work with Marius Aardal, Diego Aranha and Patrick Longa on a vectorised implementation of (2,2) isogenies
- Try and give some intuition on why this is a good idea
- Give some work-in-progress numbers to see some potential speed ups
- Not wake up my daughters zzZ

Main Goal

- SQIsign is **slow**, and the highest cost component is computing $(2^n, 2^n)$ -isogenies.
- Can we use **SIMD instructions** to get a substantial performance boost for computing these isogeny chains?
- The expected use case of SQIsign is in specialised scenarios, so maybe it's not unreasonable to recommend specialised hardware too?
- Aside from SQIsign, higher dimensional isogenies are being used constructively in many places (Prism, POKE, CORAL, Pegasus, MIKE, ...)

What are we working with

- In the $(2^n, 2^n)$ chain we have a kernel which is represented by a pair of points.
- For the first step: "gluing" we have pairs of pairs of points on elliptic curves with coordinates $(X : Z)$.
- For all other steps we have a pair of theta points on a level 2 theta structure with coordinates $(X : Y : Z : W)$.
- In both cases this means the kernel is represented by eight field elements.

Question:

How much can we vectorise doubling and evaluations working with 8 coordinates?

Doubling on the Theta Structure

```
def double(P, C1, C2):  
    """  
    Compute [2]P for a point P on OA using precomputed  
    constants Ci = (ai : bi : ci : di)  
  
    Cost: 8S + 8M + 16a  
    """  
    P = square_coordinates(P)           # 4S  
    P = hadamard(P)                     # 8a  
    P = square_coordinates(P)           # 4S  
    P = multiply_coordinates(P, C1)      # 4M  
    P = hadamard(P)                     # 8a  
    P = multiply_coordinates(P, C2)      # 4M  
    return P
```

Vectorised Doubling on the Theta Structure

```
def double(P, C1, C2):  
    """  
    Compute [2]P for a point P on OA using precomputed  
    constants Ci = (ai : bi : ci : di)  
  
    Cost: 2 4_sqr + 2 4_mul + 4 4_add  
    """  
    P = square_coordinates(P)           # 1 four-way square  
    P = hadamard(P)                     # 2 mix add sub  
    P = square_coordinates(P)           # 1 four-way square  
    P = multiply_coordinates(P, C1)      # 1 four way multiplication  
    P = hadamard(P)                     # 2 mix add sub  
    P = multiply_coordinates(P, C2)      # 1 four way multiplication  
    return P
```

Vectorised Doubling on the Theta Structure

```
def double_pair(P, Q, C1, C2):  
    """  
    Compute [2]P, [2]Q for a pair of  
    points P, Q on OA using precomputed  
    constants Ci = (ai : bi : ci : di)  
  
    Cost: 2 8_sqr + 2 8_mul + 4 8_add  
    """  
    P, Q = square_coordinates(P, Q)           # 1 eight-way square  
    P, Q = hadamard(P, Q)                     # 2 mix add sub  
    P, Q = square_coordinates(P, Q)           # 1 eight-way square  
    P, Q = multiply_coordinates(P, C1, Q, C1)  # 1 eight way multiplication  
    P, Q = hadamard(P, Q)                     # 2 mix add sub  
    P, Q = multiply_coordinates(P, C1, Q, C1)  # 1 eight way multiplication  
    return P, Q
```

Evaluation of Theta Points

```
def theta_eval(P, inv, h1, h2):  
    """  
    Evaluate a theta point through an isogeny given  
    precomputed constants inv = (a : b : c : d).  
  
    Cost 4S + 4M + 8a  
    """  
    if h1:  
        P = hadamard(P)                # 8a  
        P = square_coordinates(P)      # 4S  
        P = hadamard(P)                # 8a  
        P = multiply_coordinates(P, inv) # 4M  
    if h2:  
        P = hadamard(P)                # 8a  
    return P
```

Theta Isogenies are Vector "friendly"

- When computing the chain we have a kernel consisting of a pair of points
- Given 8-way parallelism, we can double or evaluate both points **simultaneously** batching eight fp2 operations together.
- Codomain computations can also be nicely vectorised but the code is less beautiful

Vectorised Gluing

Most of the chain very easily vectorises but the gluing isogeny requires more work.

Gluing consists of three main stages:

1. Doubling the kernel to recover the eight, four and two torsion below
2. Computing a change of basis matrix using kernel data
3. Mapping to the theta structure and evaluating the special (2,2) isogeny

This requires $4(n - 1)$ calls to `x_double()`, an expensive call for a matrix computation and then a similar cost eval as standard.

Vectorised x-only Doubling

```
def x_double(X, Z, A24, C24):  
    """  
    Compute [2]P = [2](X : Z) on a Montgomery curve  
    with projective (A + 2C : 4C)  
  
    Cost: 4M + 2S + 4a  
    """  
    t0, t1 = X + Z, X - Z          # 2a  
    t0, t1 = t0**2, t1**2          # 2S  
    t2, t3 = t0 + t1, t0 - t1      # 2a  
    t0, t1 = t3 * A24, t1 * C24    # 2M  
    X2, Z2 = t0 * t1, t2 * t3      # 2M  
    return X2, Z2
```

Vectorised x-only Doubling

- A single doubling can be naively vectorised using 2-way parallelism
- For our isogeny we have four points, which can be done using 8-way parallelism
- **Optimisation**: if $C_{24} = 1$ we can save four multiplications by replacing one 8-way with a 4-way mul.

The ugly basis change is still ugly

- The basis change computation was ugly originally and vectorisation made it worse
- **Good news**: we can batch almost all the multiplications in an 8-way configuration anyway!

Theoretical Summary

- Theta doubling and evaluations are trivial to refactor to use eight way parallelism
- Codomain computations also nicely vectorise but usually with four way parallelism
- Gluing is distinct (as always) but we can still use eight way parallelism for the majority of the field operations
- The expected speed up will be the benefit of eight way parallelism with some cost of bookkeeping, memory management and rotations of the blobs

Show us some numbers!

Fp2 Arithmetic Costs (Work in Progress)

Variant	Mul (cycles)	Square (cycles)	Mul Speedup	Sqr Speedup
64-bit	264	158	-	-
8 way	438	387	4.82×	3.27×
4 way	361	256	2.93×	2.47×
2 way	225	224	2.35×	1.41×

SQIsign Level 3 Verification Chain (Work in Progress)

- SQIsign2D (Broadwell optimised) 7,348,460 cycles
- AVX-512 implementation: 2,574,036 cycles
- **Speed up 2.85x**

SQIsignAVX

- We are planning on implementing all of SQIsign verification with our AVX-512 instructions
- Verification can be split up approximately three main parts:
 - i. Challenge 2^n Isogeny (25% cost)
 - ii. Kernel generation (12.5% cost)
 - iii. Verification (50% cost)
- The kernel generation is dominated by three calls to biscalar double ladders.
- The dimension one isogeny has been optimised in previous work *Highly Vectorized SIKE for AVX-512* by Hao Cheng, Georgios Fotiadis, Johann Großschädl and Peter Y. A. Ryan

Make Everything Dimension Two!

- Craig Costello has a lovely paper about using the Weil restriction to compute dimension one isogenies over $\mathbf{F}_{p^2}$ using a two dimensional isogeny over $\mathbf{F}_p$
- Maria Corte-Real Santos and Krijn Reijnders revisited this using Scholten's construction and squared Kummer surfaces
- **Stretch Goal:** if there's a higher relative performance boost for dim 2 versus dim 1 isogenies in vectorised implementations, we could compute the challenge isogeny using these tricks adapted for the theta model

Questions?

Recent hobby has been working on a rust library `isogeny_rs` for isogeny based cryptography. It's in early stages but im interested in collaborating to expand the toolkit.

I currently have

- Efficient arithmetic over F_p and F_{p^2} for generic modulus
- Velu and VeluSqrt for dimension one isogenies
- (2,2) isogenies in the theta model
- Cubical Pairings
- SQIsign Verification

We are missing

- Anything which looks like quaternions
- Dimension four isogenies
- Group action things (CSIDH is a WIP in a PR right now)